

# E-Stage DZ Backend: Production Readiness Checklist

This document outlines the required changes to transition the E-Stage DZ backend from a local development environment to a production-ready state suitable for cloud hosting.

## 1. Database Migration (Critical)

Currently, the application uses SQLite ( `provider = "sqlite"` in `prisma.schema` ). Cloud hosting providers (Render, Heroku, etc.) use ephemeral file systems, meaning the local `dev.db` file will be completely wiped upon every deployment or server restart.

### Required Action:

- Change the Prisma provider to `"postgresql"` or `"mysql"` .
- Provision a managed cloud database (e.g., Supabase, Neon Tech, Railway, or AWS RDS).
- Update the `.env` file to use the new `DATABASE_URL` .
- Run `npx prisma migrate dev` to generate the new schema tables.

## 2. Cloud File Storage (Critical)

The current setup uses `multer.diskStorage` to save avatars and PDF resumes directly into the server's `./uploads` directory. Similar to the database issue, these files will be permanently deleted when the cloud server restarts.

### Required Action:

- Remove `multer.diskStorage` .
- Integrate a cloud storage solution such as **Cloudinary**, **AWS S3**, or **Supabase Storage**.
- Use `multer-storage-cloudinary` or `multer-s3` to pipe uploads directly to the cloud bucket.
- Save the returned Cloud URL to the Prisma database instead of `http://localhost:5000/uploads/...` .

## 3. Pagination for Large Datasets (High Priority)

The endpoints `GET /api/jobs` , `GET /api/applications` , and `GET /api/admin/users` currently query and return all records at once without limits.

### Required Action:

- Implement pagination using Prisma's `take` and `skip` parameters.
- Accept `?page=1&limit=20` query parameters in the Express routes.
- Return metadata in the response (e.g., `totalPages` , `currentPage` ) so the frontend can implement "Load More" or pagination buttons.

## 4. Security & Rate Limiting (Medium Priority)

Currently, the `/api/auth/login` and `/api/auth/register` endpoints have no brute-force protection.

### Required Action:

- Install `express-rate-limit` .
- Apply a strict rate limit to the `/api/auth/*` routes (e.g., maximum 5 login attempts per 15 minutes per IP).

- Ensure CORS is strictly configured to only accept requests from the production frontend domain (e.g., `https://e-stage.dz` ) rather than allowing all origins or just `localhost` .

## 5. Input Validation (Medium Priority)

The Express routes currently destructure `req.body` directly into Prisma without rigorous validation, making the app susceptible to malformed data or missing required fields.

### Required Action:

- Integrate a validation library like **Zod** or **Joi**.
- Create validation schemas for User Registration, Job Posting, and Profile Updates to ensure robust data integrity before it reaches the database.

## 6. Environment Variables (Required for Deployment)

Ensure the following variables are strictly defined in the production environment:

- `DATABASE_URL` : The new managed PostgreSQL/MySQL string.
- `JWT_SECRET` : A highly secure, randomly generated string (do not use default 'supersecretkey' in production).
- `PORT` : Cloud providers dynamically assign ports. Ensure `server.js` uses `process.env.PORT || 5000` .
- `SMTP_*` : Valid SMTP credentials are required for `/api/tickets/contact` to function without throwing 500 errors.